

FCM I Programming Assignment 3

Charles Ohanian

November 26, 2024

1 Executive Summary

In this report, we explore the implementation, behavior, and performance of iterative methods for solving linear systems, focusing on their theoretical and empirical characteristics. This report comprises three distinct components, each designed to deepen understanding of iterative solvers through practical experimentation and analysis. Firstly, Richardson's First Order Stationary Method, Steepest Descent, and Conjugate Gradient are tested within the eigen-coordinate framework for symmetric positive definite matrices. Leveraging known eigen-decompositions, we analyze error and residual norms relative to theoretical damping bounds and explore the performance implications of matrix spectral properties. Comparisons across methods are made based on convergence rates, computational effort, and observed deviations from expectations. Next, three stationary methods, Jacobi, Forward Gauss-Seidel, and Symmetric Gauss-Seidel are analyzed. Convergence properties for dense matrices of various structures are examined for each method. Using spectral radii and matrix norms of the iteration matrices, we predict and assess method performance, documenting cases of convergence and divergence under different problem configurations. Finally, these analyses are extended to large, sparse symmetric positive definite matrices. By implementing efficient sparse matrix-vector products and preconditioned solves, we investigate how sparsity and diagonal dominance influence convergence and computational cost. Empirical results are analyzed in light of theoretical expectations and spectral properties.

2 Statement of the Problem

Efficiently solving linear systems $Ax = b$ is a fundamental challenge in numerical linear algebra. Iterative methods have emerged as essential tools for solving such systems, especially when the matrix A is large and sparse. Unlike direct methods, such as Gaussian elimination, which can require extensive memory, iterative methods work by refining an initial guess for x through repeated application of a computationally inexpensive update formula. This makes them particularly well-suited for large-scale problems where storing or factorizing A directly is impractical.

Iterative methods, including Richardson's First Order Stationary Method, Steepest Descent, and Conjugate Gradient, are widely used because they exploit structural properties of A , such as symmetry, sparsity, and positive definiteness, to achieve efficient convergence. These

methods require only matrix-vector products, enabling scalable solutions for problems where A is stored in compressed formats. Their convergence depends on properties like the condition number of A , the distribution of eigenvalues, and the choice of initial guess, which must be carefully analyzed to ensure numerical stability and efficiency.

Stationary iterative methods, such as Jacobi, Gauss-Seidel, and Symmetric Gauss-Seidel, provide another class of techniques for solving $Ax = b$. These methods decompose A into easily invertible components and iteratively solve simplified subproblems, often yielding faster convergence for diagonally dominant or well-conditioned matrices. However, their performance is highly sensitive to matrix ordering and sparsity patterns, requiring systematic study to understand their behavior. This report investigates the behavior of these iterative methods under varying problem conditions

3 Description of the Mathematics

Iterative methods are powerful numerical techniques used to solve large systems of linear equations, particularly when direct methods are computationally expensive. These methods approximate the solution through successive refinements, guided by the properties of the iteration scheme.

3.1 Iterative Methods

Richardson's method iteratively refines an initial guess X_0 of the solution x for $Ax = b$ using the formula:

$$x_{k+1} = x_k + \alpha r_k,$$

where $r_k = b - Ax_k$. Convergence behavior can be analyzed through the residual $r_k = b - Ax_k$ or through the error $e^{(k)} = x_k - x$, where $Ae^{(k)} = -r_k$. In this method, the choice of α determines convergence. Through the relationships $e^{(k)} = Ge^{(k-1)} = \dots = G^k e^{(0)}$ and $r_k = Gr_{k-1} = \dots = G^k r_0$, where $G = (I - \alpha A)$, it can be observed that the method will converge if $\rho(G) < 1$, where $\rho(G)$ is the spectral radius of G . If $\rho(G) > 1$, the method is not convergent, meaning there exists at least one x_0 for which the sequence $\{x_k\}$ does not converge to $x = A^{-1}b$. Hence, α must be chosen carefully in order to ensure convergence. If A is a symmetric positive definite matrix with $\lambda_{min} > 0$ and $\alpha > 0$, then convergence is guaranteed if $\alpha < \frac{2}{\lambda_{max}}$. Further computations yield that the optimal α is $\alpha_{opt} = \frac{2}{\lambda_{min} + \lambda_{max}}$. With α

chosen in this way, it can be shown that $\|e^{(k)}\|_2 \leq \frac{\kappa - 1}{\kappa + 1} \|e^{(k-1)}\|_2$ or equivalently, $\|e^{(k)}\|_2 \leq \left[\frac{\kappa - 1}{\kappa + 1} \right]^k \|e^{(0)}\|_2$, where $\kappa = \lambda_{max}/\lambda_{min}$.

Steepest descent is a nonstationary iterative method that begins with an initial guess x_0 and improves it step by step by moving in the direction where the error decreases the fastest, which is the negative gradient of the associated error function. The update step is:

$$x_{k+1} = x_k + \alpha_k r_k,$$

where $r_k = b - Ax_k$ and $\alpha_k = \frac{r_k^T r_k}{r_k^T A r_k}$. Note that α_k is set based on solving a linear least squares problem on every step. $r_{k+1} \perp r_k$, but no global relationship is maintained between all of the residual vectors. This method works by continually reducing the error, moving closer to the true solution. However, if A is poorly conditioned, the method can converge slowly because it may oscillate rather than directly approach the true x . Similarly to Richardson's method, it can be shown that $\|e^{(k)}\|_A \leq \frac{\kappa - 1}{\kappa + 1} \|e^{(k-1)}\|_A$, where $\kappa = \lambda_{\max}/\lambda_{\min}$.

The conjugate gradient method improves upon steepest descent by ensuring that each step moves in a direction that avoids undoing progress made in previous steps, i.e., it makes sure that successive residuals are conjugate, or orthogonal, with respect to A . It solves symmetric positive definite systems efficiently and converges in at most n steps for an n -dimensional system. The error after k iterations satisfies:

$$\|e^{(k)}\|_A \leq 2\alpha^k \|e^{(0)}\|_A,$$

where $\alpha = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}$. This method is widely used for large, sparse systems due to its superior convergence properties.

3.2 Stationary Methods

In order to discuss the stationary methods Jacobi, Forward Gauss-Seidel, and Symmetric Gauss-Seidel, it is useful to decompose A into $A = D - L - U$, where D is the diagonal of A , and $-L$ and $-U$ are the strict lower and upper parts of A , respectively. All three stationary methods discussed have the following form of update step:

$$x_{k+1} = x_k + P^{-1} r_k,$$

where $r_k = b - Ax_k$ and P is specific to the method:

1. Jacobi: $P = D \implies P^{-1} = D^{-1}$
2. Forward Gauss-Seidel: $P = (D - L) \implies P^{-1} = (D - L)^{-1}$
3. Symmetric Gauss-Seidel: $P = (D - L)D^{-1}(D - U) \implies P^{-1} = (D - U)^{-1}D(D - L)^{-1}$

The Jacobi method is the simplest stationary method, but it tends to converge slower than the other methods. The Forward Gauss-Seidel method improves upon Jacobi by using updated values of $x_i^{(k+1)}$ as soon as they are computed. This reliance on updated values means fewer iterations are typically needed to achieve the same accuracy compared to Jacobi. The Symmetric Gauss-Seidel method combines a forward and backward sweep for each iteration. This symmetry improves the method's robustness and it converges more reliably than the other methods. Jacobi is not guaranteed to converge for a symmetric positive definite matrix A , while both Forward Gauss-Seidel and Symmetric Gauss-Seidel do converge for SPD A . The error at the k -th iteration is given by $e^{(k+1)} = Ge^{(k)}$, where $G = I - P^{-1}A$ is the iteration matrix. Convergence requires that the spectral radius $\rho(G) < 1$, which ensures that $e^{(k)} \rightarrow 0$ as $k \rightarrow \infty$.

3.3 Sparse Matrices and Data Structures

Sparse matrices, characterized by a high proportion of zero entries, arise naturally in iterative methods. Direct methods, like Gaussian elimination, become impractical for large sparse systems because they often introduce fill-in nonzero entries in positions that were initially zero. This sparsity is leveraged to reduce storage space and computational cost. The Compressed Sparse Row (CSR) format stores only nonzero entries and their positions. CSR consists of three arrays:

1. AA, a values array that stores all nonzero values row by row.
2. JA, a column indices array that stores the column indices of the corresponding values.
3. IA, a row pointer array that stores the starting index of each row in the values array.

Compressed Sparse Column (CSC), a format that focuses on column-oriented storage can also be used. This compact representation enables efficient matrix-vector products, crucial for iterative methods.

4 Description of the Algorithm and Implementation

All routines and their subsequent testers were developed and run in Spyder, an open source python environment included within Anaconda.

4.1 Part 1 Algorithms

The three algorithms described in this section are designed to solve linear systems of the form $Ax = b$. In this implementation, the matrix A is represented by the diagonal matrix Λ , which is provided as a vector of eigenvalues. The inputs for each algorithm include Λ (diagonal eigenvalues), $\tilde{b}$ (the right-hand side vector), x_0 (an initial guess for the solution), $\tilde{x}$ (the true solution vector, used here to compute relative errors), a convergence threshold ϵ , and a maximum number of iterations. The algorithms each output the approximated solution x , the number of iterations performed, and a list tracking relative errors over the iterations.

At each iteration of the Richardson's method algorithm, the algorithm updates the solution x using the formula $x_{\text{next}} = x + \alpha r$, where $\alpha = \frac{2}{\lambda_{\min} + \lambda_{\max}}$ is a step size chosen to optimize convergence (hence α is constant throughout the iterations), and r is the residual vector $r = \tilde{b} - \Lambda x$. The residual is then updated as $r_{\text{next}} = r - \alpha \Lambda r$, and its norm is checked against the initial residual to determine if the convergence criterion $\|r_k\|/\|r_0\| < \epsilon$ is met. Additionally, the algorithm tracks the relative error $\frac{\|x_{\text{next}} - \tilde{x}\|_2}{\|x - \tilde{x}\|_2}$ at each step. The iterations stop either when the convergence criterion is satisfied or when the maximum number of iterations is reached.

At each iteration of the Steepest Descent algorithm, the residual r is computed as $r = \tilde{b} - \Lambda x$, representing the steepest descent direction. The optimal step size α_k is dynamically

calculated using $\alpha_k = \frac{r_k^T r_k}{r_k^T v_k}$, where $v_k = \Lambda r_k$. The solution is updated as $x_{\text{next}} = x + \alpha_k r$, and the residual is updated as $r_{\text{next}} = r - \alpha_k v$. The A -norm of the error, $\|x_{\text{next}} - \tilde{x}\|_A$ is tracked and used to compute the relative error, $\frac{\|x_{\text{next}} - \tilde{x}\|_A}{\|x - \tilde{x}\|_A}$, at each iteration. The iterations terminate when the residual norm satisfies $\|r_k\|/\|r_0\| < \epsilon$ or when the maximum number of iterations is reached.

At each iteration of the Conjugate Gradient algorithm, the algorithm computes a search direction d (initialized as equal to $r = \tilde{b} - \Lambda x$), chosen to be conjugate to all previous search directions with respect to A . The step size α_k is computed dynamically as $\alpha_k = \frac{r_k^T r_k}{d_k^T \Lambda d_k}$. The solution is updated as $x_{\text{next}} = x + \alpha_k d$, and the residual as $r_{\text{next}} = r - \alpha_k v$. The relative error $\frac{\|x_{\text{next}} - \tilde{x}\|_A}{\|x - \tilde{x}\|_A}$ is tracked to assess convergence. The search direction is updated using $d_{\text{next}} = r_{\text{next}} + \beta d$, where $\beta = \frac{r_{\text{next}}^T r_{\text{next}}}{r^T r}$, ensuring conjugacy. The iterations terminate when the residual norm satisfies $\|r_k\|/\|r_0\| < \epsilon$ or when the maximum number of iterations is reached.

Efficiency is achieved in all three algorithms by exploiting the diagonal structure of Λ , reducing matrix-vector multiplications to element-wise operations, which require $O(n)$ operations, instead of $O(n^2)$ for dense matrices. The algorithms also perform updates in-place for x and r to minimize memory usage. In the steepest descent method, the dynamic computation of α ensures that the algorithm takes the most effective step toward reducing the residual at each iteration. By leveraging conjugate directions, the conjugate gradient method typically converges in fewer iterations compared to the other two methods, making it highly efficient for large-scale systems.

4.2 Part 2 Algorithms

In this section, a function is described that encapsulates the three main algorithms. However, first note that two algorithms from programming assignment 2 are utilized here: `Lv_solver` and `Uv_solver`, which compute lower triangular solves and upper triangular solves, respectively. These algorithms remain the same as in the last assignment, with the added relaxation that the lower triangular matrix is not necessarily unit lower triangular.

The stationary method function contains three algorithms for solving linear systems of the form $Ax = b$ through stationary iterative methods. The inputs for the function include A (a dense matrix), b (the right hand side vector), x_0 (an initial guess for the solution), $\tilde{x}$ (the true solution vector for error tracking), `method` (a flag specifying which stationary method to use), a convergence threshold ϵ , and a maximum number of iterations. In each of the three algorithms, the residual is computed as $r = b - Ax$. Then the algorithms differ due to the distinct preconditioner used in each method. In the Jacobi algorithm, the solution is updated as $x_{\text{next}} = x + D^{-1}r$, where D is the diagonal of A . Since the inverse of a diagonal matrix is given by just replacing each diagonal element with its reciprocal, D^{-1} can be computed

as $(1/\text{np.diag}(A)) * r$ (note that the $*$ operator can be used here as both $(1/\text{np.diag}(A))$ and r are stored as vectors and thus all that's needed is component-wise multiplication). In the Gauss-Seidel algorithm, the solution is updated as $x_{\text{next}} = x + (D - L)^{-1}r$, where $(D - L)$ is the lower triangular part of A (including the diagonal). The algorithm calculates $(D - L)^{-1}r$ as $\text{Lv_solver}(L, r)$ (where $L = \text{np.tril}(A)$ is the lower triangular part of A), which computes a lower triangular solve $Lv = r$ for v using forward substitution. In the Symmetric Gauss-Seidel algorithm, the solution is updated as $x_{\text{next}} = x + (D - U)^{-1}D(D - L)^{-1}r$, where $(D - U)$ is the upper triangular part of A , $(D - L)$ is the lower triangular part of A (both including the diagonal), and D is the diagonal of A . The algorithm first calculates $z_1 = (D - L)^{-1}r$ in the same way as in the Gauss-Seidel algorithm. Then, $z_2 = Dz_1$ is computed as $\text{np.diag}(A) * z_1$ (the $*$ operator can again be used here as we have component-wise multiplication of two vectors). Then $z_3 = (D - U)^{-1}z_2$ is computed as $\text{Uv_solver}(U, z_2)$ (where $U = \text{np.triu}(A)$ is the upper triangular part of A), which solves the upper triangular system $Uv = r$ for v using backwards substitution. Then, we have $x_{\text{next}} = x + z_3 r$. In addition, all three algorithms compute the relative error $\frac{\|x_{\text{next}} - x_{\text{true}}\|_2}{\|x_{\text{true}}\|_2}$ at each iteration and check this against the convergence threshold ϵ .

Each method leverages the structure of A to optimize computations. Decomposing A into $A = D - L - U$ allows for the preconditioners to be computed using direct vector-vector multiplication or triangular solves. The update in Jacobi only requires $O(n)$ computations since it is only component-wise multiplication of two vectors. The updates in Gauss-Seidel and Symmetric Gauss-Seidel require $O(n^2)$ operations since they involve triangular solves. Additionally, the in-place update of x in each method minimizes memory usage.

4.3 Part 3 Algorithms

The first algorithm in part 3 generates a random sparse symmetric positive definite matrix. The user specifies a dimension n , the range of values for matrix elements (a, b) , the sparsity level, and a diagonal multiplier to ensure positive definiteness. First, an $n \times n$ zero matrix is initialized. Then, for each row i , a limited number of nonzero elements are randomly chosen in earlier columns, respecting the sparsity level. These values are mirrored across the diagonal to ensure symmetry. Random values are assigned to the diagonal elements and finally, the diagonal is boosted to ensure each row's diagonal value is sufficiently larger than the sum of the magnitudes of the other elements in that row, ensuring positive definiteness. In terms of efficiency, the algorithm efficiently limits the number of nonzeros by leveraging sparsity. The symmetry ensures fewer operations during storage and processing.

The second algorithm in this part converts the lower triangular portion of a sparse matrix A into the compressed sparse row (CSR) format. Three lists are initialized, `aa` (to store the values), `ja` (to store the column indices), and `ia` (to store the row pointers). A loop iterates through each row, adding nonzero elements to `aa` and their respective indices to `ja`. The row pointer `ia` is updated accordingly. Storing a sparse matrix in CSR format significantly reduces memory usage compared to storing the entire dense matrix. Moreover, the symmetry of the matrix is exploited by only storing the lower triangular part. From this, the upper triangular part can also be extracted.

The third algorithm in part 3 computes the product of a sparse matrix A (stored in CSR format) with a vector x . The diagonal of A is stored separately in D . Firstly, the solution vector z is initialized. For each row i , the algorithm finds the diagonal element $D[i]$ and calculates the contributions from lower triangular nonzero elements and the upper triangular nonzero elements (through symmetry). Using CSR format avoids unnecessary operations on zero entries, optimizing performance for large sparse systems.

The fourth algorithm in this part solves a system of equations $Lz = b$, where L is a sparse lower triangular matrix stored in CSR format. Firstly, the solution vector z is initialized with zeros. Then the first element $z[0]$ is computed directly using the diagonal. For each subsequent element $z[i]$, the algorithm calculates the dot product of the already computed values in z with their corresponding coefficients in L , subtracts this from $b[i]$ and divides by the diagonal element $D[i]$. This forward substitution method is efficient for sparse matrices, with complexity proportional to the number of nonzero elements in L .

These algorithms, along with a sparse upper solve algorithm should be used to update the six algorithms from parts 1 and 2 to handle sparse SPD matrices stored in a compressed format. Unfortunately, after days of futile attempts, this did not come to fruition.

5 Description of the Experimental Design and Results

5.1 Description of Tester for Part 1

In order to test the algorithms for Richardson's method, Steepest Descent, and Conjugate Gradient for a wide variety of problems, five distinct problem types were created, each of them differing in how Λ , the vector of eigenvalues, is generated. The five problem types are as follows:

1. All n eigenvalues the same
2. k distinct eigenvalues with multiplicities chosen deterministically
3. k distinct eigenvalues with random distributions around each distinct eigenvalue
4. Eigenvalues chosen from a uniform distribution
5. Eigenvalues chosen from a normal distribution

When the user runs the tester, they input which problem type they would like to test. After selecting the problem type, the user can choose the size of the matrix (i.e. the number of eigenvalues), n , the number of distinct eigenvalues or means, k , if testing problem type 2 or 3, and a maximum and minimum value for the eigenvalues. Thus, many combinations of problem type, matrix size, etc. can be tested. For the problem type selected, the tester generates a random true solution vector $\tilde{x}$ and computes the associated $\tilde{b} = \Lambda\tilde{x}$. Then, ten random initial guess vectors, x_0 , are generated and for each, the tester runs each of the three iterative methods using the generated Λ , $\tilde{x}$, $\tilde{b}$, and x_0 . Each test creates three plots:

1. A comparison of the iterations until convergence using the different methods for each of the ten initial guesses.

2. A plot that shows the relative error at each step of the Richardson's and steepest descent algorithms, $\frac{\|e^{(k+1)}\|}{\|e^{(k)}\|}$, being bounded above by $\frac{\kappa - 1}{\kappa + 1}$. Due to the differing convergence results, the relative error in Richardson's is computed using the 2-norm, while the relative error in steepest descent is computed using the A -norm. All ten initial guess tests for both methods are combined on the same plot for convenience because they share an upper bound.
3. A plot that shows the relative error at each step of the Conjugate Gradient algorithm, $\frac{\|e^{(k)}\|_A}{\|e^{(0)}\|_A}$, being bounded above by $2\alpha^k$ where $\alpha = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}$. These error ratios are plotted separately from those of the other two algorithms because this upper bound is dependent on the number of iterations k .

5.2 Results for Part 1

For each of the five problem types, three varying matrix sizes were tested:

1. $n = 10, k = 3$.
2. $n = 100, k = 15$.
3. $n = 1000, k = 90$.

Note that k is only relevant for problem types 2 and 3. In all tests, each of the eigenvalues are between 10 and 100. This range was chosen to ensure reasonable condition numbers.

For problem type 1, all three tests ($n = 10, 100, 1000$) produced the same results. With all three methods, the tests convergence in one step in every case. The error plots are also trivial as there is only one iteration. For these reasons, the plots for this case are omitted. Importantly, these results align with the theory. Since there is only one distinct eigenvalue, all three methods can converge to within ϵ of the solution vector in just one step.

Figure 1 displays the results of problem type 2. The iteration count plots show that for all dimensions, steepest descent converges at least as fast as Richardson's and conjugate gradient converges at least as fast as steepest descent, which aligns with the theory. When $n = 10$ and $k = 3$, conjugate gradient converges in 3 iterations for every test case. This is evidence of Theorem 9.10 in Set 9, which says that if a matrix A has m distinct eigenvalues, then CG converges in at most m steps. This is also backed up in the larger dimension tests, as CG always converges in at most k steps. The error ratio plots show that the error ratio $\frac{\|e^{(k+1)}\|}{\|e^{(k)}\|}$ (using the 2-norm in Richardson's and the A -norm in SD) are bounded above by $\frac{\kappa - 1}{\kappa + 1}$, which is backed up by the theory in Set 8. Interestingly, as the dimension increases, the time it takes the error ratio to converge to this bound also increases. This makes sense as an increased number of eigenvalues would theoretically cause the errors at each step to converge slower, which would cause the ratio to increase slower. In addition, as the dimension increases, it becomes notable that the error ratios for Richardson's increase to the bound

slightly faster than those for SD. The conjugate gradient error also stays below the specified bound at every iteration until convergence.

Figure 2 displays some results from problem type 3. As dimension increases, the plots follow a similar progression as those in Figure 1 (problem type 2). Accordingly, and for space and convenience, only the plots for $n = 1000$, $k = 90$ are included. When comparing convergence rates between the different methods, similar behavior can be seen, with steepest descent converging at least as fast as Richardson's and conjugate gradient converging at least as fast as steepest descent. In problem type 3, there are still n distinct eigenvalues, but they are concentrated in k clusters. This should mean that conjugate gradient converges in not necessarily k steps, but at least a decent amount faster than n steps. This is validated by the results. For the $n = 100$, $k = 15$ test, CG converges in about 22 steps for each initial guess. When $n = 1000$, $k = 90$, CG converges actually in significantly less than k steps, converging in about 20 iterations. Very similar behavior to problem type 2 is seen in the error ratio plots. The error ratios stay below the upper bounds, converging to the bound slower for higher dimension levels.

Figure 3 displays some results from problem type 4. The plots again follow a similar progression as dimension increases. For this reason, only the plot for $n = 100$, $k = 15$ is included in the report. In this problem type the eigenvalues are chosen randomly from a uniform distribution, so there can be n distinct eigenvalues. For all test cases, CG converges in 20 steps, significantly lower than the upper bound of 100. Steepest descent and Richardson's method both converge in many more steps than CG, but SD always converges faster than Richardson's. Looking at the error ratio plots, the results back up the theory, as the error ratios always remain bounded by the specified upper bounds. It is also somewhat noticeable that the error ratios for Richardson's increase to the bound slightly faster than those for SD before converging.

Figure 4 displays some results from problem type 5. In this problem type the eigenvalues are chosen randomly from a normal distribution, so there can be n distinct eigenvalues. However, the methods should converge faster than they did in problem type 4, because the values are likely closer together than when chosen from a uniform distribution. The results back up this intuition, as the number steps until convergence is lower in problem type 5, even at a higher dimension. For the $n = 1000$ test, although there can be 1000 distinct eigenvalues, CG converges in less than 20 iterations for every test case. In this test, it can also be observed that there are two test cases in which the number of iterations for Richardson's and steepest descent matches, highlighting that SD is not always strictly better than Richardson's, even if it tends to be in general. Similar behavior can be observed in the error ratio plots as with the other problem types. The theoretical upper bounds are empirically confirmed and it is noticeable that as dimension increases, the error ratios for Richardson's and SD converge to the bound slower.

5.3 Description of Tester for Part 2

In order to test the algorithms for Jacobi, Forward Gauss-Seidel, and Symmetric Gauss-Seidel, nine matrices (A_0 through A_8) with various properties are considered. From here,

the tester can be broken down into two parts. In the first part, the user inputs which method and which matrix they would like to test. The tester then computes the iteration matrix G for their chosen matrix and chosen stationary method. The spectral radius $\rho(G)$ and the matrix 2-norm $\|G\|_2$ are then computed and printed. The program also computes and prints the number of iterations until convergence k with a b for a solution $\tilde{x}$ that is all 1's and an initial guess $x_0 = 0$. If the method did not converge in 10,000 iterations, it was marked as not convergent (N/A in the table). For each test matrix and stationary method, useful information can be derived from $\rho(G)$, $\|G\|_2$, and k . In the second part of the tester, the user selects a matrix to test and 20 tests are run with all 3 stationary methods. Each test consists of a random solution $\tilde{x}$ and its corresponding b and a random initial guess x_0 . The entries in both $\tilde{x}$ and x_0 are floats between -50 and 50. For each of the 20 test cases, the iterations until convergence are plotted for each method. Iterations were limited to 250 for the sake of the plots, so if a system does not converge in 250 iterations, it is assumed to not converge.

5.4 Results for Part 2

Table 1 displays the results of the first part of the tester. It shows the spectral radius $\rho(G)$, the matrix 2-norm $\|G\|_2$, and the number of iterations until convergence k for each matrix using each stationary method. It can be seen that A_0 does not converge for any of the three methods. This aligns with what we would expect based on the spectral radii and matrix norms seen in the table, as they are quite a bit greater than 1. With A_1 , $\rho(G_J) > 1$ and $\rho(G_{GS}) = \rho(G_{SGS}) < 1$. Consequently, we see that Jacobi does not converge, while Gauss-Seidel and Symmetric Gauss-Seidel both converge in 12 iterations. Meanwhile, the tests of A_2 showed some interesting behavior, where Jacobi and SGS converged, while GS did not. If we observe the spectral radii, this again aligns with the theory as $\rho(G_J)$ and $\rho(G_{SGS})$ are less than 1, while $\rho(G_{GS}) > 1$. Observing A_3 , A_4 , A_5 , and A_6 , it can be seen that all three methods converge with all of these matrices, but the data highlights some intuition about the speed of convergence. Firstly, we see that Jacobi converges with A_3 and A_4 even though $\rho(G_J) > 1$. This aligns with the theory because even if $\rho(G) > 1$, there can still exist many initial guesses for which the method converges. Additionally, we see that for all four of these matrices, the speed of convergence is directly correlated to the magnitude of both $\rho(G)$ and $\|G\|_2$. The lowest values for $\rho(G)$ in the entire table occur when using GS and SGS for A_3 and these methods converge in just 5 and 4 steps, respectively. With all of these matrices, the lower the magnitude of $\|G\|_2$ or $\rho(G)$, the lower the number of iterations until convergence. Observing A_7 and A_8 , it can be seen that both of these matrices are symmetric positive definite. That means according to the theory, they should converge with Gauss-Seidel and Symmetric Gauss-Seidel. The results back this up, as we see that these matrices actually converge with all three methods. Again, we see relationships between the magnitude of the spectral radii or the matrix norm and the speed of convergence.

Figures 5 through 12 display the results for second part of the tester. When testing A_0 with many random solutions and initial guesses, we did not come across a single case where there was convergence for any of the three stationary methods. For this reason, the plot is omitted. This aligns with the behavior seen in Table 1. Figures 5 and 6 show the plots for A_1 and A_2 , respectively. The results here exactly line up with the results in the first part, as Jacobi

never converges for A_1 and Gauss-Seidel never converges for A_2 . Figures 7 and 8 show the plots for A_3 and A_4 , respectively. These plots also line up with what we would expect from the initial test in the first part. Both matrices converge in all test cases with all methods. For A_3 , GS and SGS converge in very few iterations and for A_4 , Jacobi converges quicker than Gauss-Seidel. Figures 9 and 10 show the plots for A_5 and A_6 , respectively. Aligning with our observations from Table 1, it can be seen that these two matrices always converge with all three methods, with the speed of convergence also matching the prior test. Figures 11 and 12 show the plots for A_7 and A_8 , respectively. As predicted, since these matrices are SPD, Gauss-Seidel and Symmetric Gauss-Seidel converge with all tests. Moreover, Jacobi also converges in every test similarly to the prior part. All in all, expanding the tests to include random solution and initial guess vectors $\tilde{x}$ and x_0 reaffirmed the results displayed in Table 1 and aligned with the theory discussed.

5.5 Discussion of Tester and Results for Part 3

Although a test driver was not completed for this part, its theoretical outline and predicted results will be discussed here. The tester would begin with the user specifying which method they would like to test: Richardson's, steepest descent, conjugate gradient, Jacobi, Forward Gauss-Seidel, or Symmetric Gauss-Seidel. For each of the first three iterative methods, the driver would run a number of tests for each problem type. Each test would generate three random sparse vectors: one containing the eigenvalues Λ , one as the true solution $\tilde{x}$, and one as an initial guess x_0 , all stored in a compressed format. The tester would compute $\tilde{b} = \Lambda\tilde{x}$ and then call each of the three updated methods to test for convergence. The iteration counts would be plotted for easier comparison. For each of the three stationary methods, numerous tests would be run as well. Each test would generate a random $n \times n$ sparse matrix and store the lower triangular part with CSR format (using the algorithms described in section 4.3), as well as generate a random true solution vector $\tilde{x}$ and a random initial guess vector x_0 . After computing $b = \Lambda\tilde{x}$, the tester would call each of the three updated stationary methods to test for convergence. The Jacobi method, as in part 2, would not need any lower or upper triangular solves, but the Forward Gauss-Seidel and Symmetric Gauss-Seidel algorithms would make use of the CSR lower solve and CSR upper solve codes. The iteration counts for the many tests would be plotted for easier comparison between the methods.

It is expected that the results in this section would line up heavily with the results of parts 1 and 2. Specifically, the distribution of eigenvalues would affect the convergence speed of the iterative methods. No matter how large or sparse the matrix is, if all the eigenvalues are equal, the methods should all converge in 1 step. Furthermore, tests within problem types where the eigenvalues are clustered should converge faster than those where they are not (for example, when they are chosen from a uniform distribution). Conjugate gradient should always be found to converge in a number of steps at most equal to the number of distinct eigenvalues. Since all generated matrices will be symmetric positive definite, the stationary methods Gauss-Seidel and Symmetric Gauss-Seidel should always converge. Jacobi may converge sometimes too, but this is not guaranteed. Analysis of the spectral radii of the large sparse matrices could provide some insight into this.

6 Conclusions

This paper explored the theoretical foundations, implementation, and performance of several iterative methods for solving linear systems, providing a comprehensive analysis through both structured testing and theoretical discussion. Starting with the derivation of iteration matrices and stationary methods, key properties dictating convergence behavior were discussed, such as spectral radii and matrix norms. The analysis in part 1 conveyed a relationship between the distribution of the eigenvalues and the speed of convergence. The analysis in part 2 revealed clear relationships between spectral properties and convergence, with detailed testing on matrices of varying characteristics. Results aligned well with theoretical predictions, illustrating the importance of matrix properties such as symmetry and positive definiteness in method selection. Expanding the analysis with random solutions and initial guesses further validated these findings, emphasizing consistency across diverse test cases. In part 3, a theoretical framework was outlined for testing all six methods discussed. This work not only demonstrates the effectiveness of iterative solvers but also establishes a foundation for further exploration of their applications to broader classes of problems.

7 Figures and Tables

7.1 Part 1

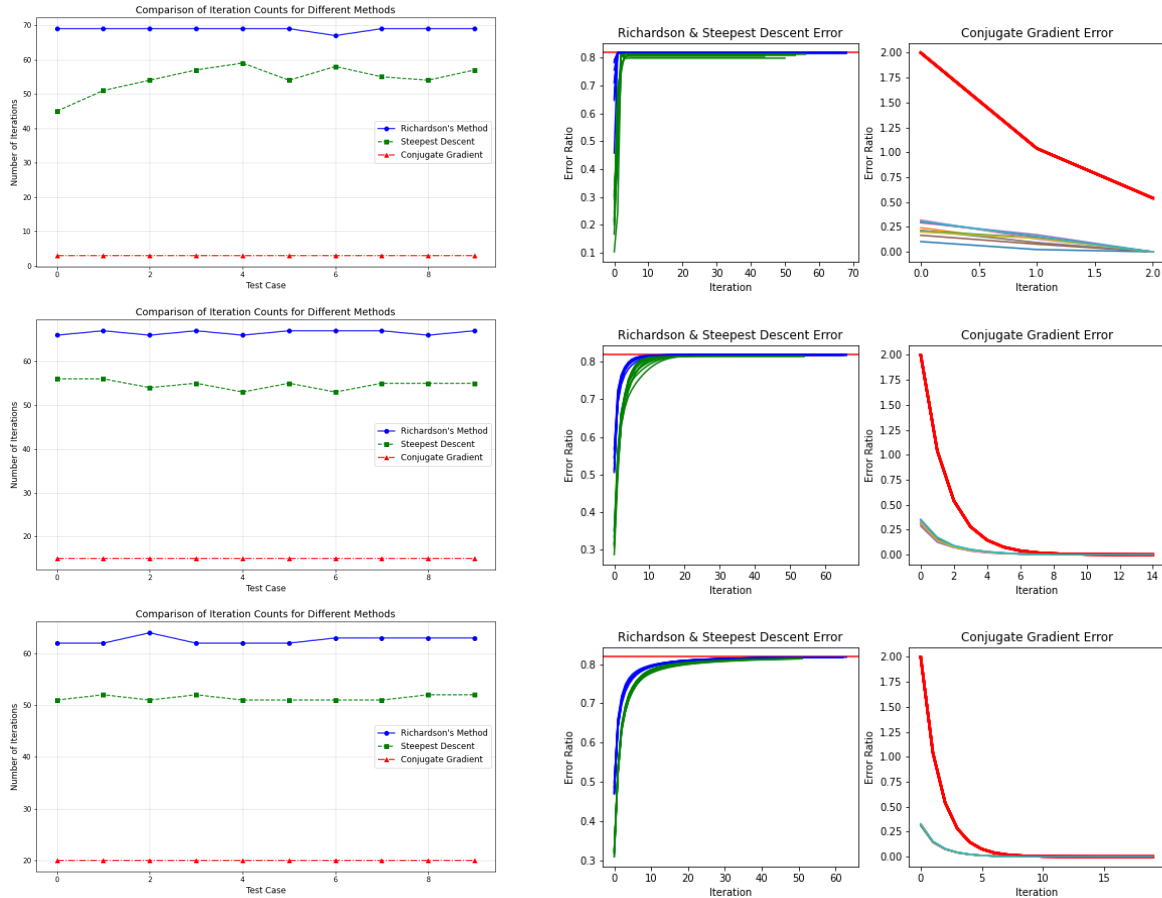


Figure 1. Results for problem type 2. Top two plots are for $n = 10, k = 3$. Middle two plots

are for $n = 100$, $k = 15$. Bottom two plots are for $n = 1000$, $k = 90$. On the Richardson's and steepest descent error ratio plots, the blue curves correspond to the Richardson's tests, while the green curves correspond to the SD tests.

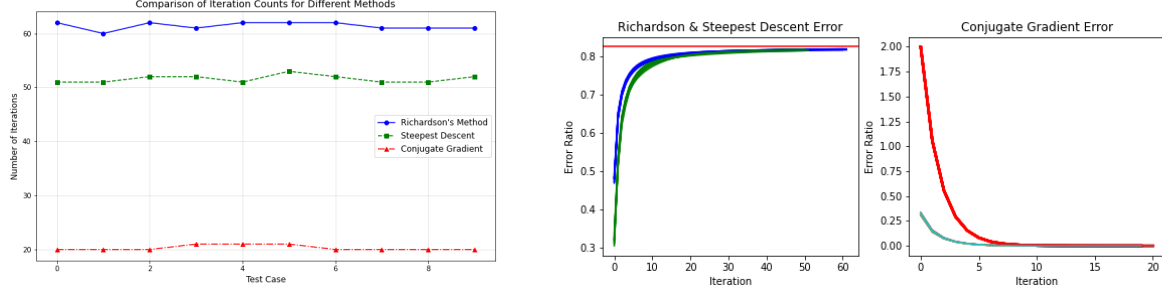


Figure 2. Results for problem type 3. Plots are for $n = 1000$, $k = 90$. The smaller dimension plots are omitted, but discussed. On the Richardson's and steepest descent error ratio plot, the blue curves correspond to the Richardson's tests, while the green curves correspond to the SD tests.

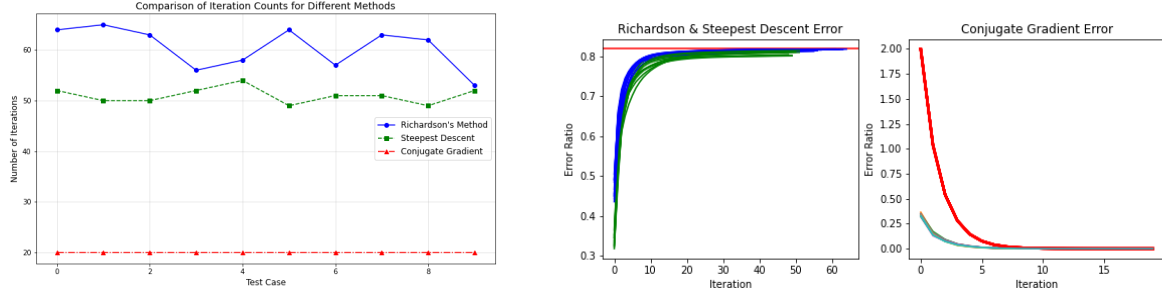


Figure 3. Results for problem type 4. Plots are for $n = 100$, $k = 15$. The plots for the other two dimensions are omitted, but discussed. On the Richardson's and steepest descent error ratio plot, the blue curves correspond to the Richardson's tests, while the green curves correspond to the SD tests.

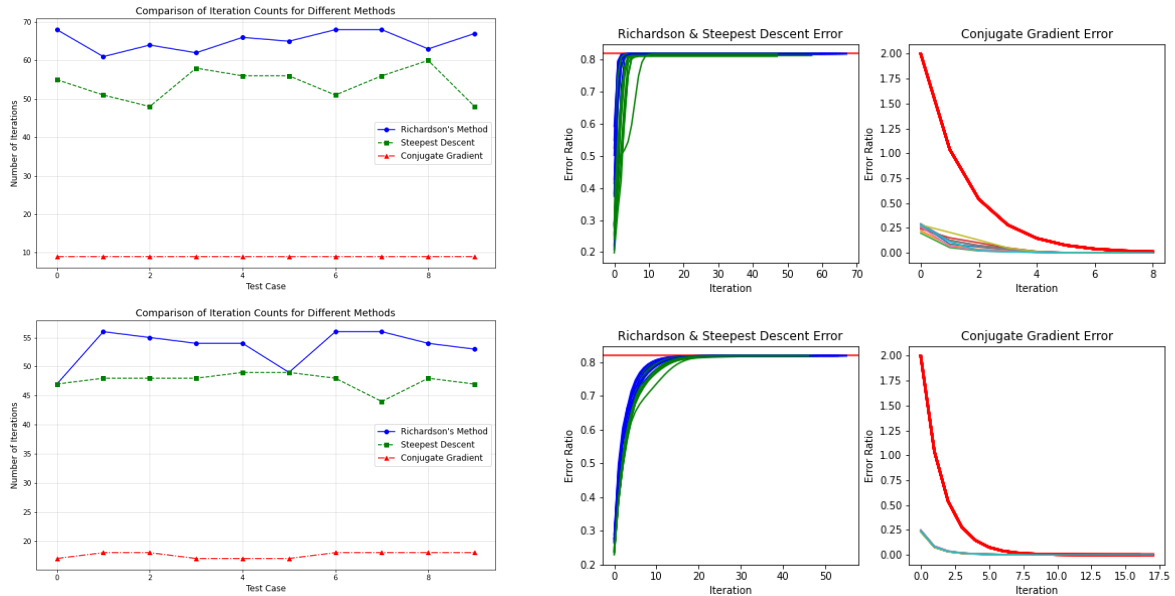
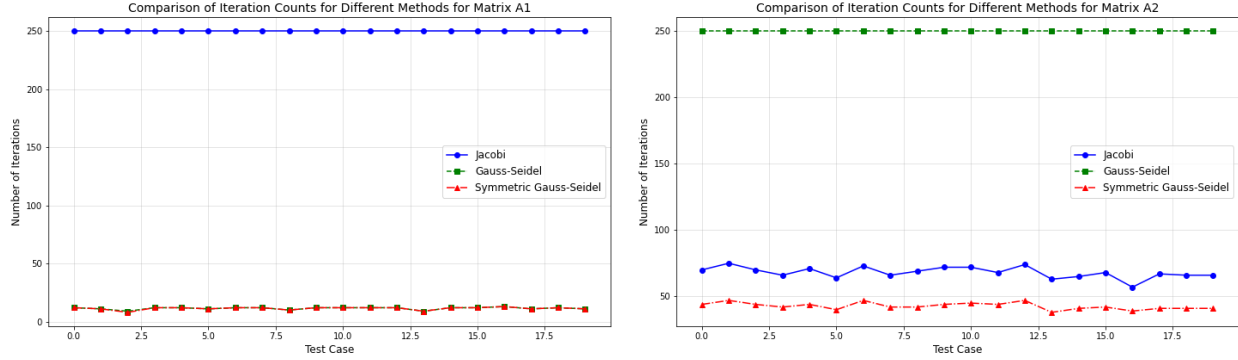
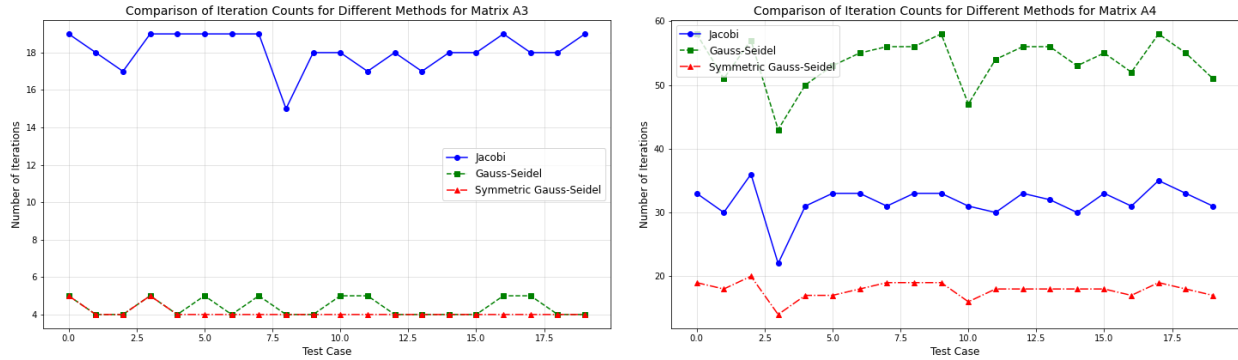


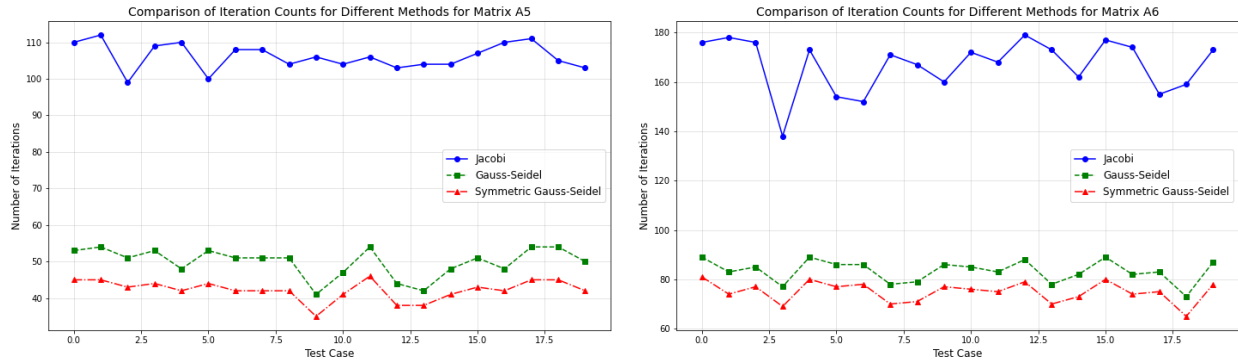
Figure 4. Results for problem type 5. Top two plots are for $n = 10$, $k = 3$. Bottom two plots are for $n = 1000$, $k = 90$. The plots for the $n = 100$ test are omitted, but discussed. On the Richardson's and steepest descent error ratio plots, the blue curves correspond to the Richardson's tests, while the green curves correspond to the SD tests.



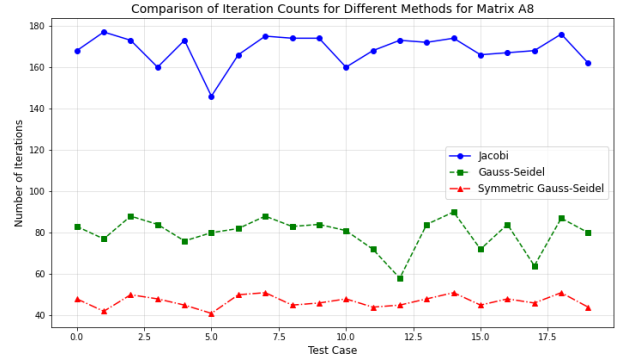
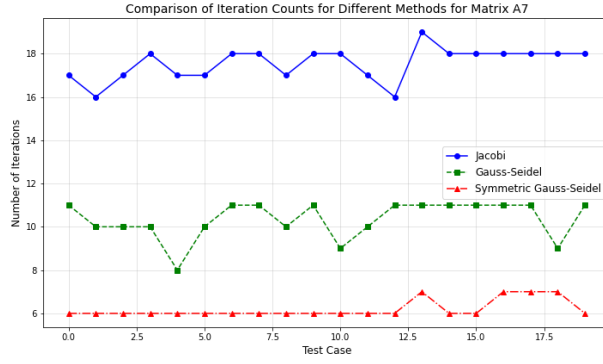
Figures 5 & 6. Iterations until convergence for A_1 (left) and A_2 (right). Maximum iterations was set at 250.



Figures 7 & 8. Iterations until convergence for A_3 (left) and A_4 (right). Maximum iterations was set at 250.



Figures 9 & 10. Iterations until convergence for A_5 (left) and A_6 (right). Maximum iterations was set at 250.



Figures 11 & 12. Iterations until convergence for A_7 (left) and A_8 (right). Maximum iterations was set at 250.

		Convergence Data								
		ρ_J	ρ_{GS}	ρ_{SGS}	$\ G_J\ _2$	$\ G_{GS}\ _2$	$\ G_{SGS}\ _2$	k_J	k_{GS}	k_{SGS}
Test Matrix	A_0	6	4.728	5.374	7.004	5.792	13.932	N/A	N/A	N/A
	A_1	4.583	0.25	0.25	5.492	2.281	2.247	N/A	12	12
	A_2	0.999	1.111	0.713	3.508	2.686	3.312	55	N/A	42
	A_3	3.611	0.019	0.019	4.313	0.377	0.131	18	5	4
	A_4	2.893	0.775	0.454	3.012	2.549	0.990	33	56	19
	A_5	0.999	0.767	0.730	1.826	0.831	0.748	105	52	44
	A_6	1	0.85	0.835	2.063	0.920	0.869	173	84	75
	A_7	3	0.213	0.103	3.114	0.322	0.103	18	11	7
	A_8	1	0.854	0.761	1.871	0.887	0.772	174	88	51

Table 1: Spectral radii, norms, and iterations until convergence for all test matrices. $x_0 = [0 \ 0 \ \cdots \ 0]^T$ and $\tilde{x} = [1 \ 1 \ \cdots \ 1]^T$.

8 References

In the creation of this report, I referenced Sets 6-10 of the class notes. I also consulted with Dan Mazus and Brayton Link about algorithm development.